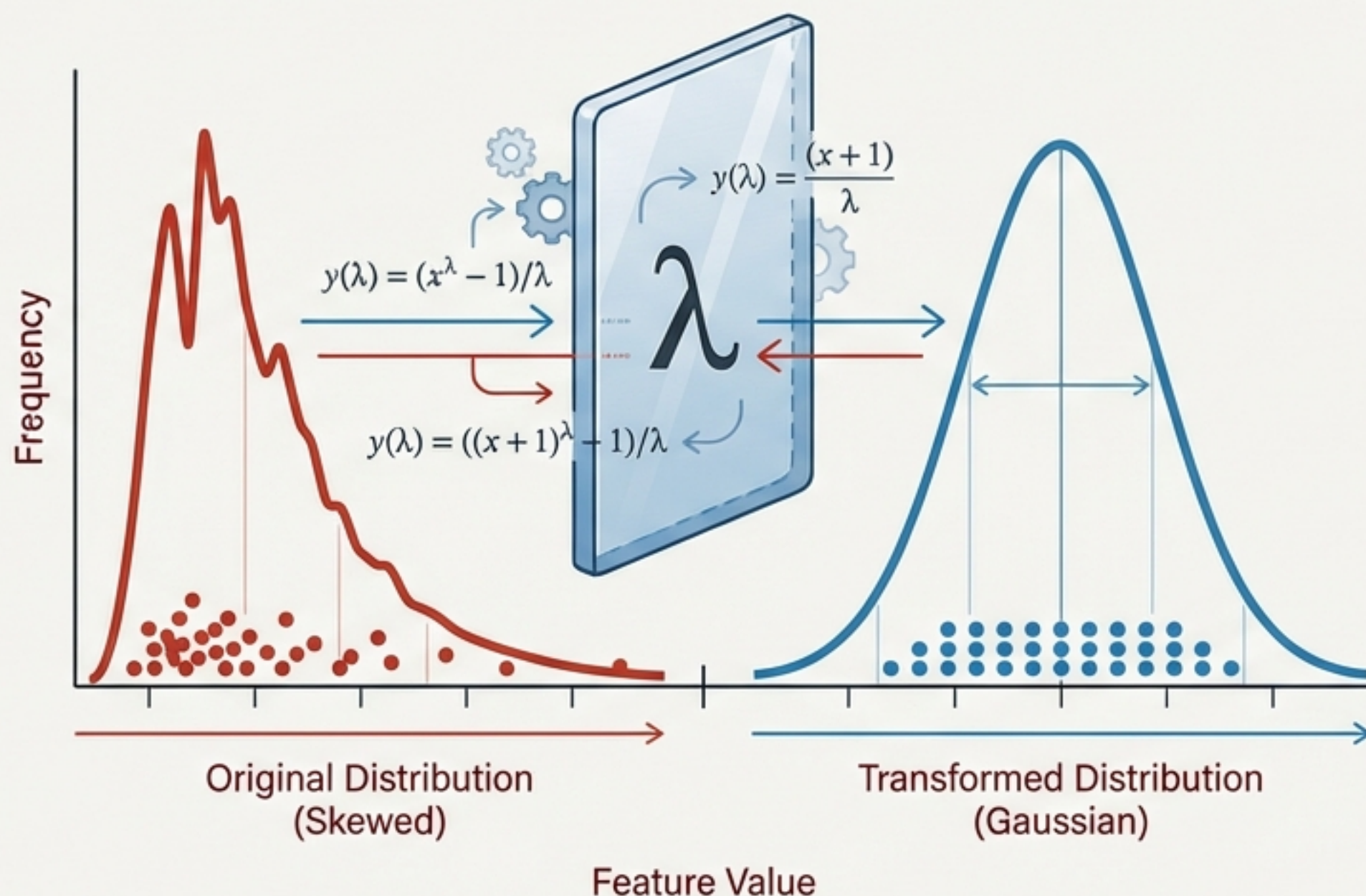


Power Transformers in Machine Learning

Mastering Box-Cox and Yeo-Johnson Transformations

Optimising feature distributions to maximise the performance of linear models.

Based on the technical analysis of the Concrete Material dataset.



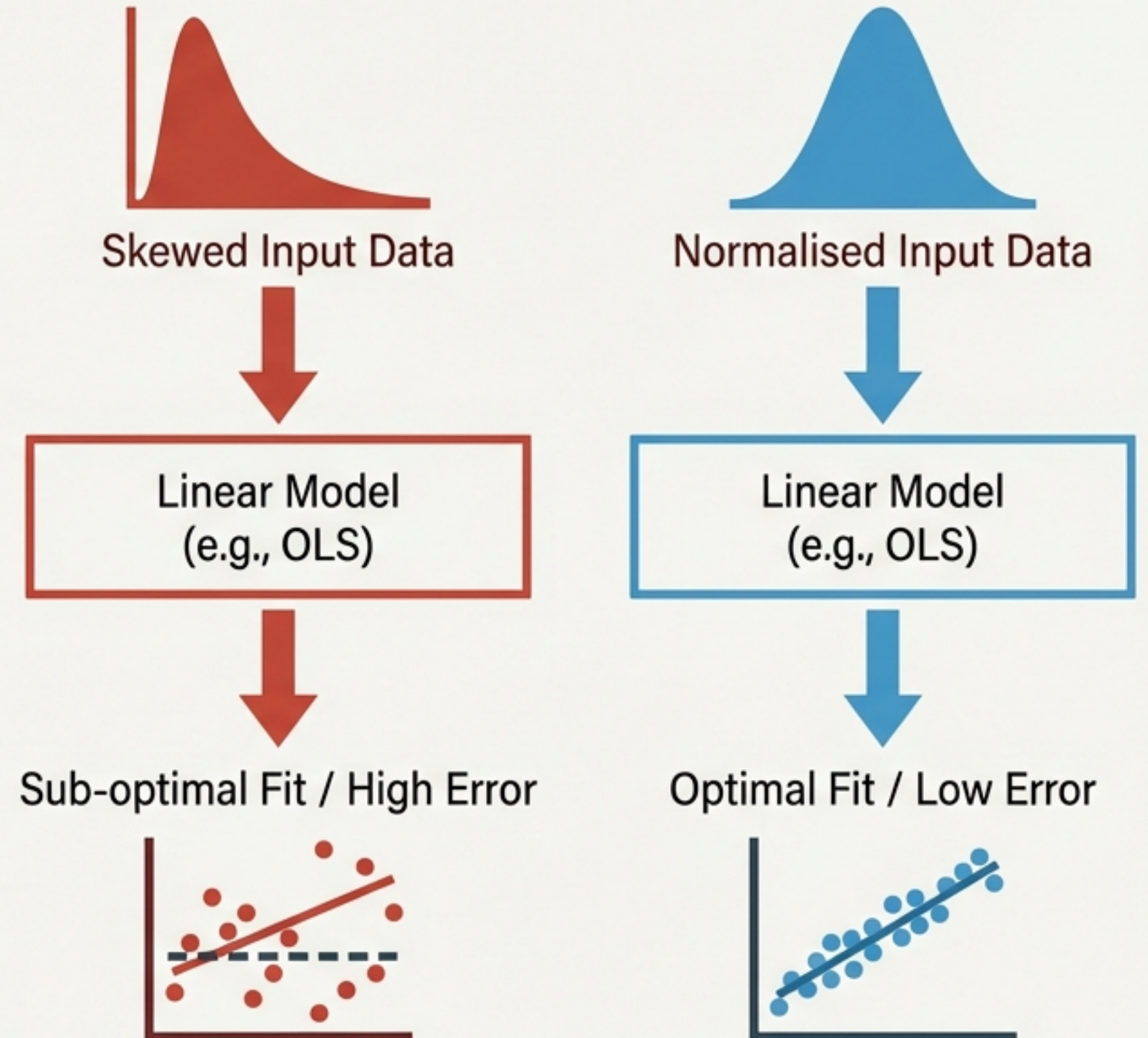
The Gaussian Assumption in Predictive Modelling

Many core machine learning algorithms operate on a fundamental assumption: that the input data follows a Normal (Gaussian) distribution. When this assumption is violated, the mathematical foundations of these models shake, and performance degrades.

Impacted Algorithms:

- Linear Regression
- Logistic Regression
- Linear Discriminant Analysis (LDA)

Key Insight: While deep learning networks may adapt to raw data distributions, classical linear models require statistically 'clean' inputs to function optimally.



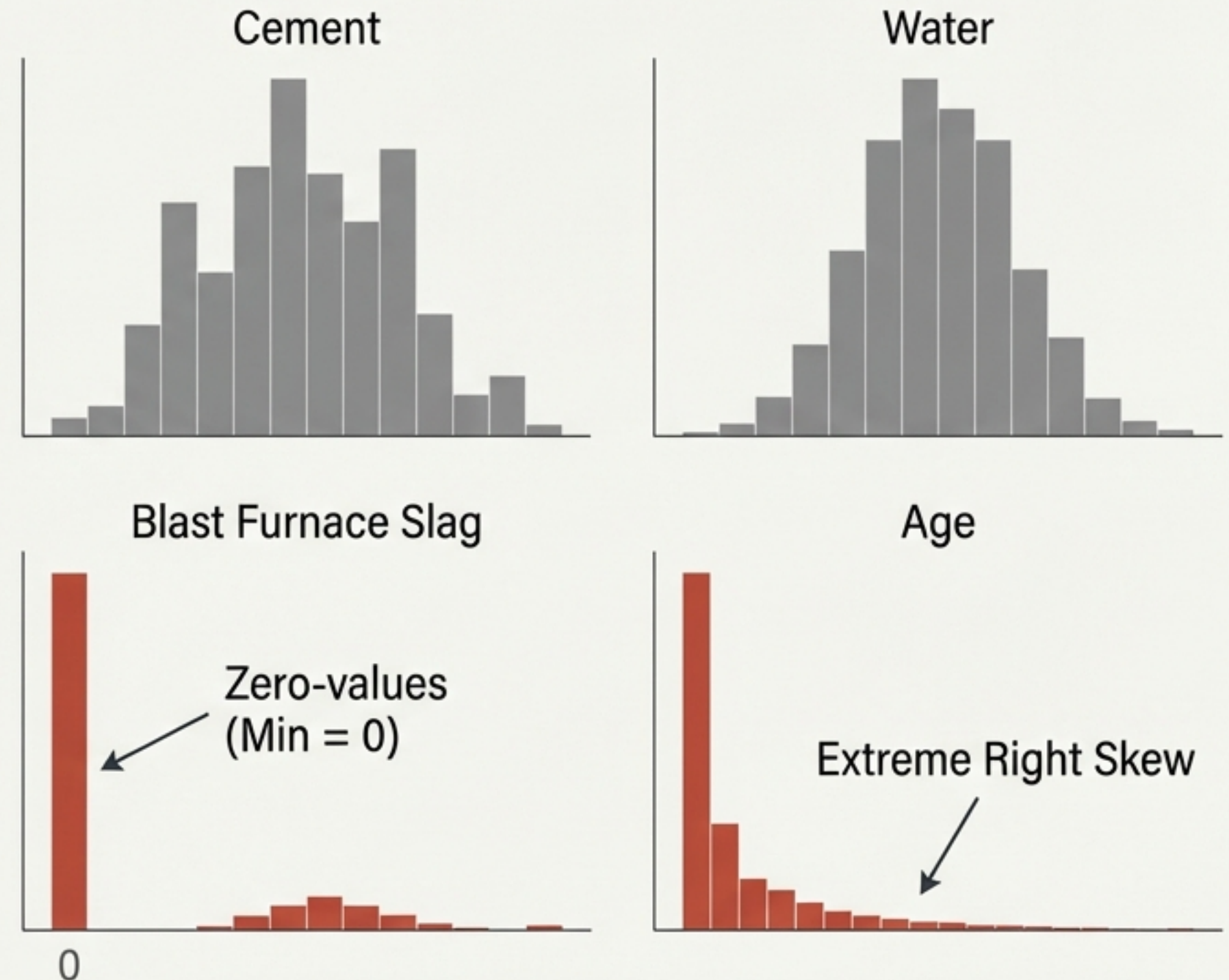
The Reality of Raw Data: Concrete Strength Case Study

Real-world data is rarely "normal". It is skewed, peaked, or flat.

In this analysis, we tackle a regression problem predicting "Concrete Strength" based on 8 input features. The raw data presents significant statistical irregularities that hinder linear modeling.

Problem Profile:

- **Skewness:** The 'Age' column is heavily right-skewed.
- **Zero-Inflation:** Columns like 'Blast Furnace Slag' contain zero values, complicating mathematical transforms.

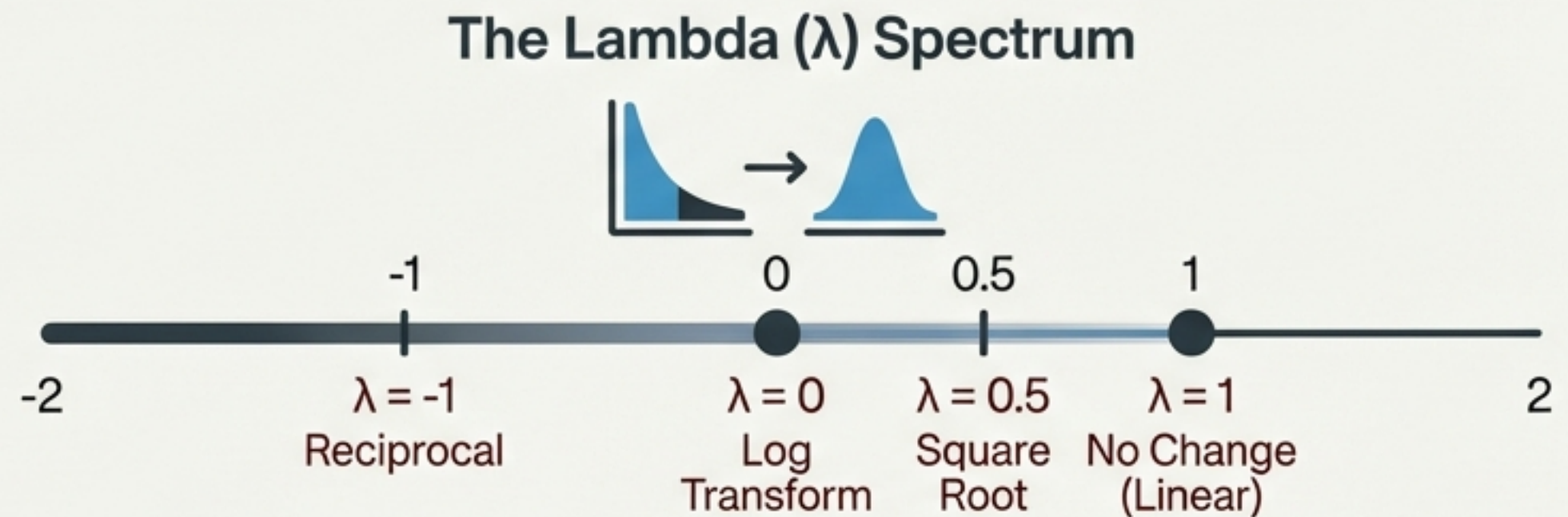


Power Transformers: A Parametric Approach

Definition: A family of parametric, monotonic transformations that map data to a Gaussian distribution to stabilise variance.

The Differentiator: Unlike fixed 'Function Transformers' (such as Log or Square Root) which apply a rigid rule, Power Transformers *learn* the optimal shape for the data.

The Mechanism: The algorithm performs a search for an optimal parameter, Lambda (λ), which acts as an exponent to transform the input features.



The algorithm slides along this scale to find the perfect exponent that makes the histogram look like a bell curve.

The Box-Cox Transformation

The foundational power transform technique, developed by statisticians Box and Cox in 1964. It is powerful but comes with a critical mathematical limitation.

The Concept

$$y(\lambda) = \begin{cases} \frac{x^\lambda - 1}{\lambda} & \text{if } \lambda \neq 0 \\ \ln(x) & \text{if } \lambda = 0 \end{cases}$$

Uses Maximum Likelihood Estimation to find λ .

The Constraint

Strictly Positive Data Only ($x > 0$)

Input Feature	
24.5	✓
10.2	✓
0.0	✗ — FAILS on Zero
-5.1	✗ — FAILS on Negatives

The Yeo-Johnson Transformation

The modern standard and default method in Scikit-Learn's PowerTransformer. Developed in 2000, it is an analytical adjustment to Box-Cox designed to handle the messiness of real-world data.

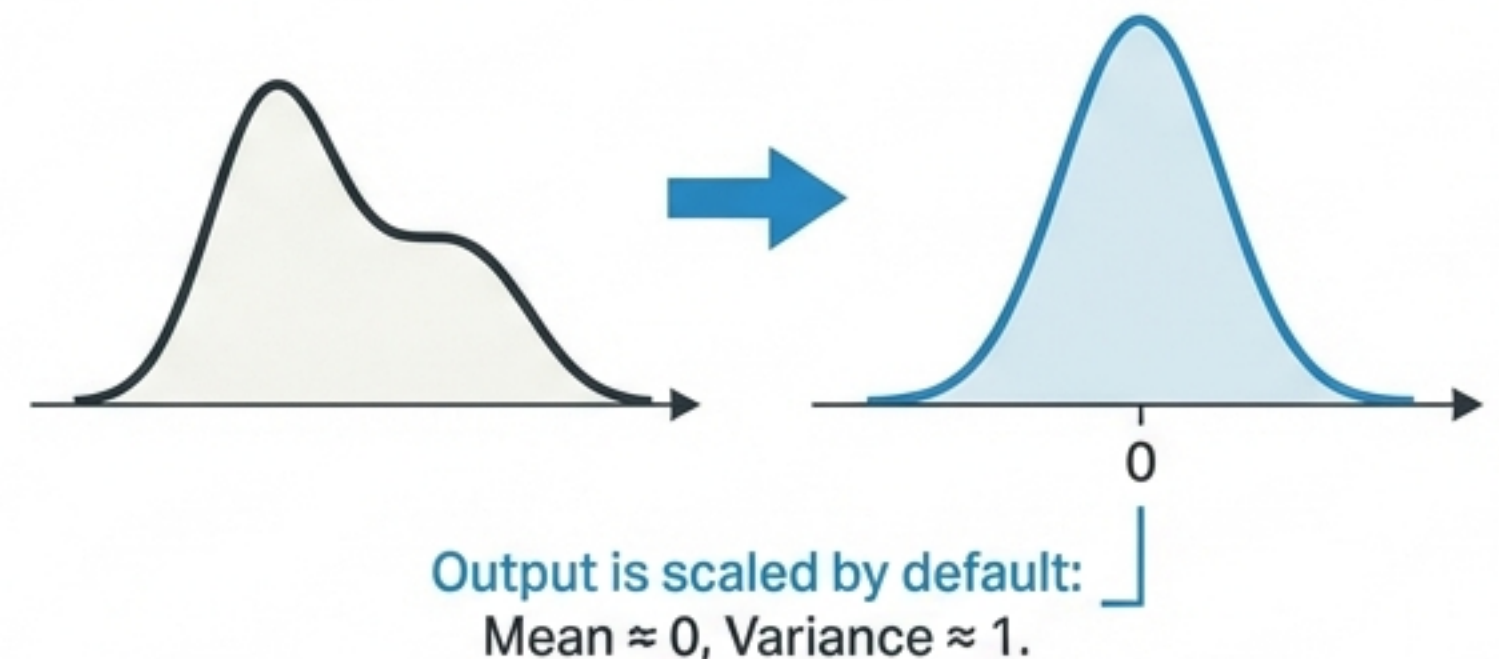
Universal Support

Input Feature	
24.5	✓
0.0	✓
-5.1	✓

HANDLES Zeros

HANDLES Negatives

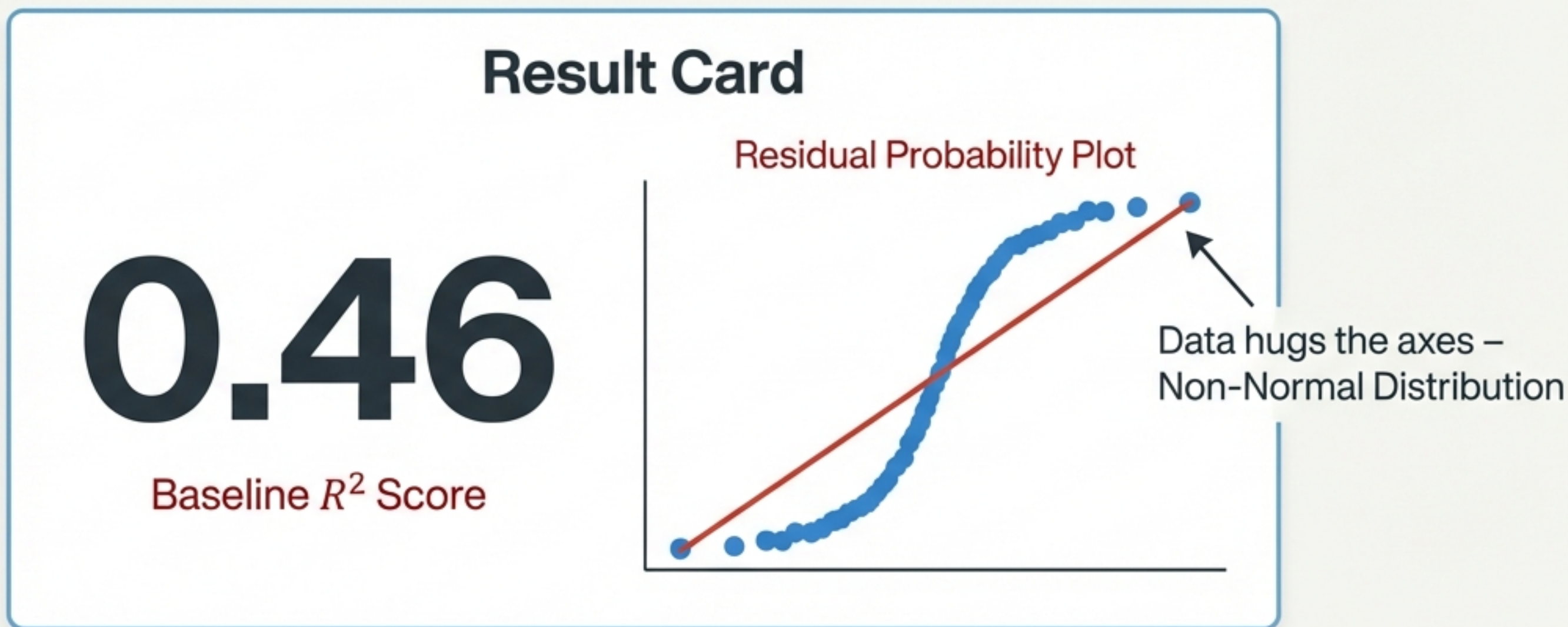
Automatic Standardisation



Establishing the Baseline: Performance on Raw Data

To measure the impact of feature engineering, we first train a Linear Regression model on the raw, un-transformed Concrete dataset.

- **Experiment:** Linear Regression (OLS)
- **Metric:** Cross-Validation R^2 Score (10 folds)



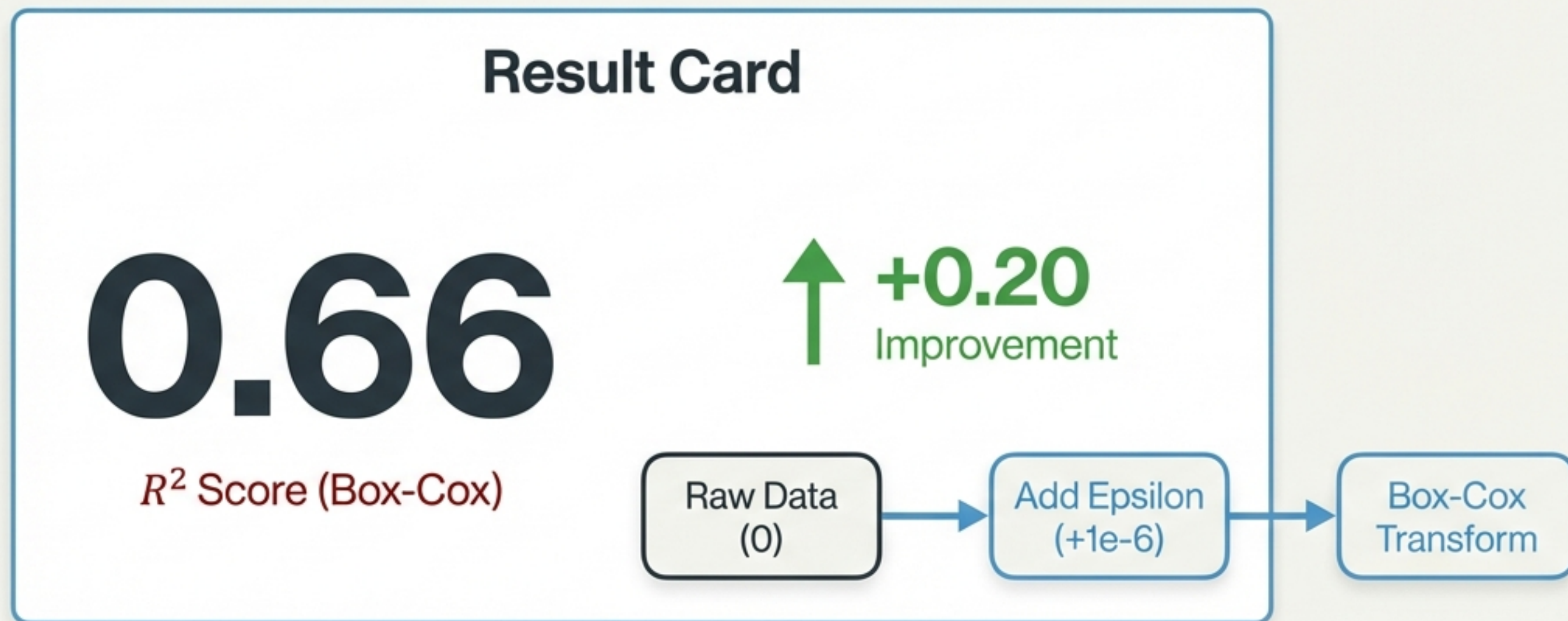
Experiment A: Applying Box-Cox

The Obstacle: The 'Blast Furnace Slag' column contains zero values. Box-Cox fails immediately with a value error.

The Workaround: We must manually add a small constant (epsilon) to shift all data to positive territory.

$$x' = x + 0.000001$$

Analysis: Effective, but requires manual intervention ('hacks').



Experiment B: Applying Yeo-Johnson

- **Implementation:** Native application via Scikit-Learn. No manual constant addition required; the algorithm handles zeros in "Slag" automatically.
- **Analysis:** The robust, "set it and forget it" solution. It slightly outperforms Box-Cox while eliminating data preprocessing hacks.

Result Card

0.68

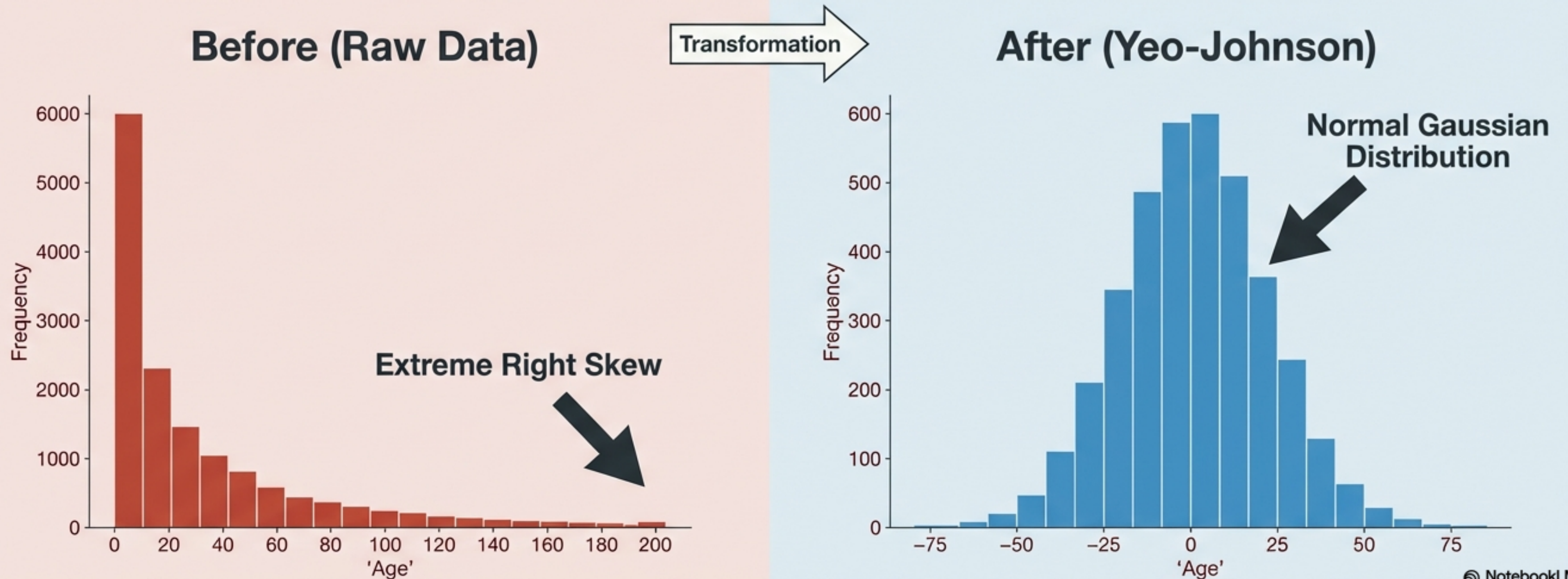
R^2 Score (Yeo-Johnson)



- ✓ Handles Zeros natively
- ✓ Auto-scaling
- ✓ Highest Accuracy

Visualising the Transformation: The 'Age' Feature

The most dramatic improvement occurred in the 'Age' column, which was heavily right-skewed.



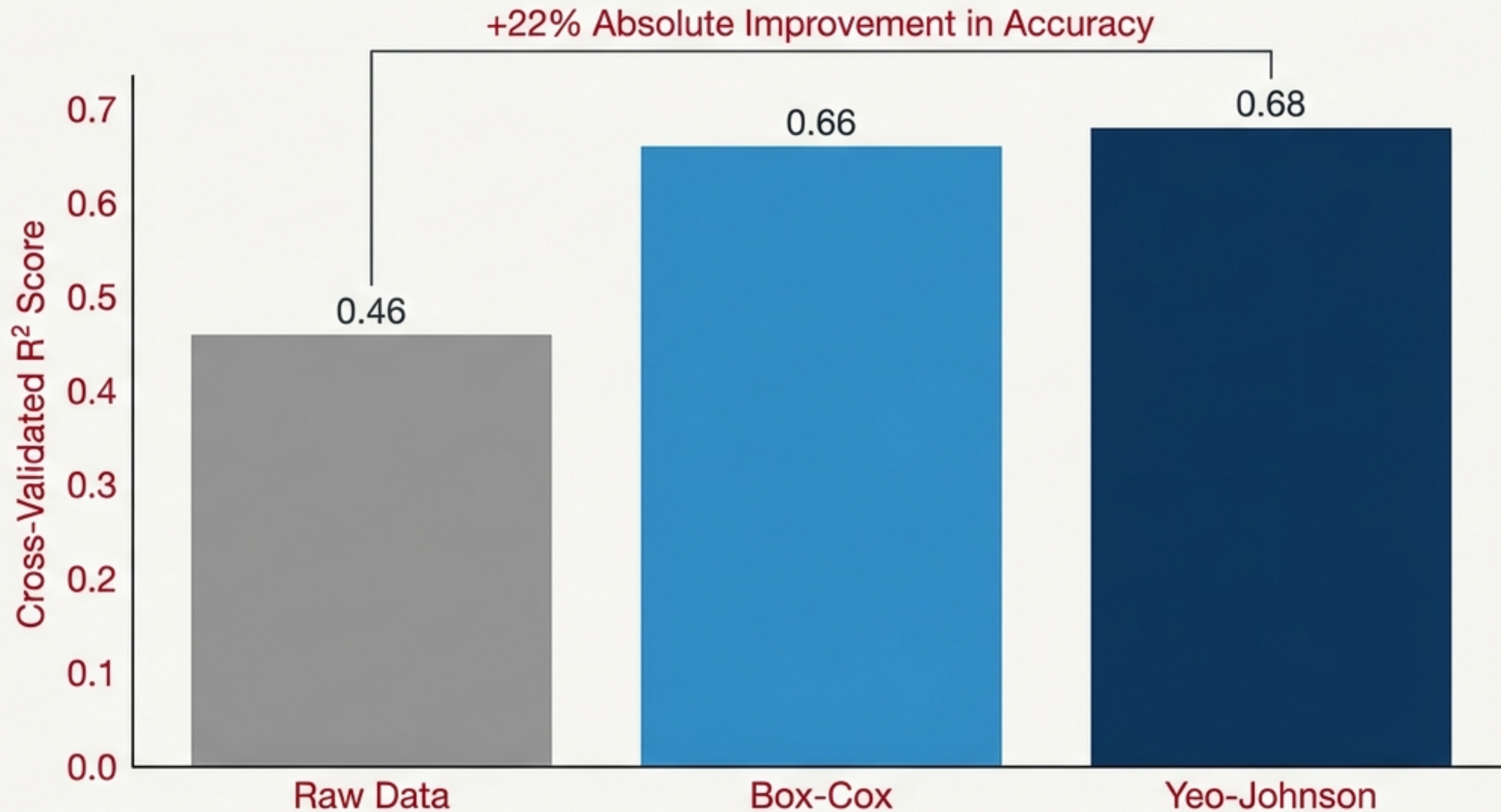
Under the Hood: The Learned Lambda (λ) Values

The PowerTransformer calculates a unique optimal λ for every single feature independently.

Feature	Learned λ	Visual Bar
Cement	$\lambda \approx 0.17$	
Blast Furnace Slag	$\lambda \approx 0.02$	 Near Log Transform
Fly Ash	$\lambda \approx -0.01$	
Age	$\lambda \approx 0.05$	

The algorithm automatically detected that ‘Slag’ and ‘Age’ required near-logarithmic transformations ($\lambda \approx 0$) to stabilise their variance.

Impact Analysis: Model Performance Summary



Feature engineering alone significantly boosted model accuracy without changing the core algorithm.

Implementation in Scikit-Learn

Python Snippet

```
from sklearn.preprocessing import PowerTransformer

# 1. Instantiate the transformer
# method='yeo-johnson' is default and supports negative values
# method='box-cox' requires strictly positive input
pt = PowerTransformer(method='yeo-johnson')

# 2. Fit and Transform the training data
X_train_transformed = pt.fit_transform(X_train)

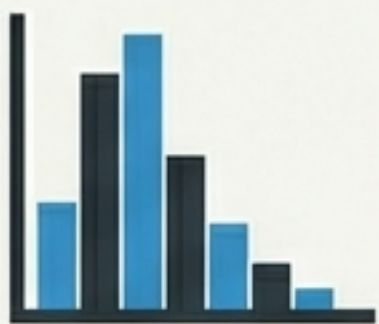
# 3. Transform the test data (using lambda learned from train)
X_test_transformed = pt.transform(X_test)
```

Pro-Tip

Output Format: The transformer returns a NumPy array. You may need to wrap it back into a Pandas DataFrame to keep column names.

Strategic Takeaways for Data Scientists

When to Use



Whenever you are using linear models (Linear Regression, Logistic Regression, LDA) and your feature histograms show visible skew or non-normality.

Box-Cox vs. Yeo-Johnson

$$x_{\lambda} = \left(\frac{rr - c_{\lambda}x}{1 - x} \right)$$

strictly positive

$$w_{eo} = \left(\frac{-1 - x^2}{1 - x} \right)$$

default/negatives

Box-Cox: Use only if data is strictly positive and you need the specific physics/economic interpretation of that transform.

Yeo-Johnson: The default choice. Handles zeros/negatives and includes standardisation.

The Bottom Line



Normal distribution isn't just a theoretical nice-to-have; it is a tangible performance booster. Expect significant gains in R^2 with minimal code overhead.